

Encapsulation: bundling data and operations together while controlling the access to internal state.

Abstraction :- It is the process of eliminating not essential details, so that only the necessary feature of the entity remain visible to the caller.

Establishes a clear boundary between the specifications and implementation.

Interfaces: An interface in java is a pure structural concept.

A class can implement multiple interfaces, by passing the single inheritance restriction of class. All fields declared inside an interface are public static variables.

All method signatures declared implicitly.

Instance variable is a field declared inside a class, outside any method, constructor or code block.

An abstract class is a class marked with the abstract keyword that cannot be directly instantiated. It acts as a partial blueprint intended to be extended by concrete subclasses.

- no curly braces
- no implementation
- enclosing class constrains.

An abstract method only exists inside an abstract class or an interface.

imp interview questions:-

abstract class vs instance class

instance variable vs concrete variable

instance variable local vs instance variable global.

Mandatory overriding:

The first concrete subclass extending an abstract parent class must override and provide concrete implementation for all inherited abstract method

```
abstract class A {  
    abstract int x, y;
```

```
}  
A(int x, int y) {
```

```
    this.x = x;
```

```
    this.y = y;
```

```
}
```

Q Create an abstract class shape that represents geometric shapes.

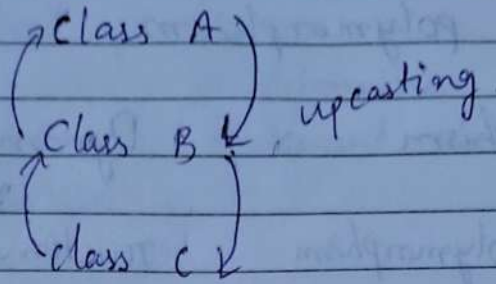
double area()

double perimeter()

void display

Inheritance: It is a mechanism where in a child class derives state and behaviour from parent class using extends keyword.

it establishes a has-a relationship



upcasting: child class acting like parent class
down casting: parent class acting like child class

this. → parameterized constructor

```

class student
{
}
student() {
}
student (string name)
this.name = name;
  
```

"This" → current object
this.student() → other constructor

Polymorphism → a greek word → means many forms

It is the capability of the single entity method calling or interface references to exhibit different variables depending on the context or the underlying object type.

two types of polymorphism

static polymorphism
↓

compile time polymorphism
↓

overloading
↓

binding time.

Dynamic polymorphism
↓

runtime polymorphism
↓

overriding
↓

Zero runtime

Compile time polymorphism - method overloading :-
method overloading occurs when a single class defines multiple methods sharing same name but having different parameters.

overloading can't be achieved solely by changing the return type of the method.

```
Public int add (int a, int b) {  
    return a+b;  
}
```

Method overriding:-

This occurs when a child class provides a specific implementation of a method that is already declared in its parent class or an interface. When calling overridden method through a parent reference, java evaluates the actual object type on the heap at runtime rather than the reference type.

Control flow :-

It determines the order of in which the statement executes, we have sequential execution.

```
int a = 10
int b = 20
int sum = a + b
S.out (sum)
```

Control flow lets a program make decisions and execute different blocks of code based on condition

There are 4 control flow

- if
- if-else
- if-else-if-ladder
- switch

if statement :-

executes only when condition is true

Syntax :-

```
if (condition)
{
    statement;
}
```

if-else :-

Provides an alternative path when the condition is false.

Syntax:-

```
if (condition) {  
    statement;  
}
```

```
} else {  
    statement;  
}
```

Public class marks {

public static void main (String [] args) {

int marks = 45;

if (marks > 35) {

System.out.println (" pass ");

} else {

System.out.println (" fail ");

}

if-else-if ladder;

checks multiple condition in sequence, only the 1st true block executes.

Syntax:-

```
if (condition 1) {
```

```
    statement;
```

```
} else if (condition 2) {
```

```
    statement;
```

```
} else if (condition 3) {
```

```
    statement;
```

```
} else {
```

```
    statement;
```

```
}
```


Eg:-

```
if (marks > 85) {  
    s.out("grade A");  
} else if (marks > 70) {  
    s.out("grade B");  
} else if (marks > 55) {  
    s.out("grade C");  
}  
elseif (marks > 40) {  
    s.out("grade D");  
}  
else {  
    s.out("fail");  
}
```

Nested if :-

An if inside another if

Switch statement:-

Switch expression is evaluated once, the result is compared with each case value.

If there is a match, the matching block of code runs,

The break statement stops at switch after the matching case has run.

The default statement runs if there is no match.

Syntax:-

```
Switch (expression) {
```

```
    Case 1:
```

```
        Sys;
```

```
        break;
```

```
    Case 2:
```

```
        Sys;
```

```
        break;
```

```
    Case 3:
```

```
        Sys;
```

```
        break;
```

```
    default;
```

Eg:-

```
int day = 3
```

```
Switch (day) {
```

```
    Case 1:
```

```
        S.out ("Monday");
```

```
        break;
```

```
    Case 2:
```

```
        S.out ("Tuesday");
```

```
        break;
```

```
    Case 3:
```

```
        S.out ("Wednesday");
```

```
        break;
```

```
    default:
```

```
        S.out ("invalid");
```

```
}
```

loop:-

It executes a block of code repeatedly based on a condition

for loop:-

used when the number of iteration known in advance.

while:-

The number of iteration isn't known before hand.

continue:-

skips the next iteration.

break:-

exits the loop at mentioned iteration

Methods: A method is a reusable block of code that encapsulates a single responsibility

Recursion:

Recursion occurs when a method calls itself to solve a problem by working on at smaller instance of the same problem

Java runtime uses a call stack to track method calls

Every call pushes a new frame on to the stack when a method returns, it's frame is popped and control returns to the caller.

Generics lets classes, interfaces, and methods operate on a type parameter specified by the caller giving compile time, type safety and eliminating manual casting.

y based

Collection framework
Set of classes and interfaces that provide readymade data structure to store and manipulate ~~the~~ group of objects.

collection is an object that can store multiple element as a single unit.

Collection provides interface define what a data structure can do.

Implementations :- Actual class that implement those interfaces is ~~also~~ called implementation.

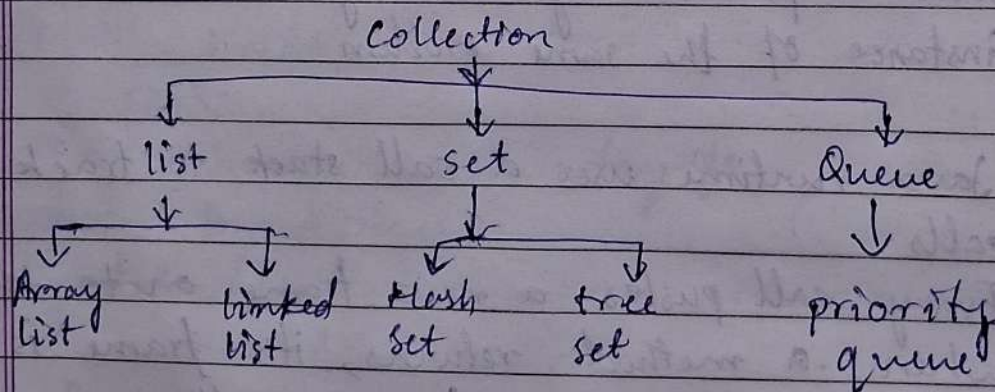
Iterators:-

mechanism for traversing collection.

collection-interface

collection-utility class

In collection we use list, set, queue.



Wrapper class:

Wrapper class is a class whose object wraps or encapsulates data types.

byte < Byte>

int < Integer>

char < Character>

float < Float>

list interface: A list represent an ordered collection maintains insertion orders, allows duplicates, supports index based access, elements can be accessed using index

Array list is one of the most frequently used java collection in DSA

Syntax:- `ArrayList<datatype> name = new ArrayList<>();`

An arraylist is backed internally by dynamic array. Unlike a normal array, it can grow dynamically.

Write a program to manage student marks using arraylist: 85, 92, 78, 90, 65, 88, 76

Find the and display the highest, lowest, average and remove all mark below 80, display after removing, check if 90 is present.

Linked list

Linked list implements 3 interfaces at once, which are list, Queue, Dequeue. which is why it shows up in so many different roles internally it is a doubly-linked list. Each node stores 3 things, reference to previous node, data, reference to the next node.

Array list

Dynamic Array
fast

less memory overhead
 $O(1)$ insertion

linked list

doubly-linked list
slow

more memory overhead
 $O(1)$ insertion

Multithreading:-

A process of executing multiple threads concurrently within a single program.

Thread:- thread is the smallest unit of execution within a process

UI thread

Network thread → when network doesn't connect.

Rendering thread → process of network being connected single thread.

Process Vs thread

A process is a running program with its own independent isolated block of memory that other process cannot directly see into.
Eg: chrome

Thread is an execution path within a process

If everything happens sequentially, the application may appear slow. The user is stuck waiting for the download to finish completely before processing event starts and waiting for processing to finish before the UI updates at all.

Event though these are conceptually, fairly independent activities

The Thread can be created by using Extends and implements.

Tree map keeps its keys in sorted order at all times. Like tree set, tree map implements a navigable interface with the same family of neighbour finding methods and operating on keys.

map.floorKey(25) → highest

map.ceilingKey(25) → lowest

map.firstKey(25) → first element

map.lastKey(25) → last element

map.headMap(25) → from top

map.tailMap(25) → from bottom

Comparable: It defines a class's single natural ordering implemented inside the class itself.

Comparator: - It defines an external swapping ordering useful when you need several different over ordering for the same class

An exception is an abnormal condition that occurs during the execution of a program and disrupts the normal expected flow of instructions.

Throwable:-

It is the superclass of all objects that can be thrown by java exception. Every error and every exception checked or unchecked ultimately extends throwable. It provides methods such as getMessage(), toString(), getCause() → SQL Exception

Set interface :-

A set interface represents a collection of unique elements

Set interface extends the collection interface
if it is available in the java.util package
It doesn't allow duplicate elements

Syntax :-

~~Set <Integer> name = new HashSet <> ();~~
Set <datatype> name = new HashSet <> ();

Use a HashSet when you need fast membership and uniqueness

Tree set :-

Tree set stores ~~uniqueness~~ elements in sorted order.

set <datatype> name = new TreeSet <> ();

Queue Interface

offer (); :- add

poll (); :- remove

peek (); :- returns 1st element without removing.

Maps :- Maps store data as key value pairs

Key must be unique.

Map <datatype, datatype> name = new HashMap <> ();

Q Create a simple student result management system where student Roll no. is key, student name + marks are value. Perform adding, searching, updating, ~~its~~ removing and displaying.

Eg:- `for(int i=0; i<=5; i++){`
`}`

Time complexity : $O(n)$

Asymptotic analysis:

The time required by an algorithm comes under 3 cases:-

Worst:- more time - big O

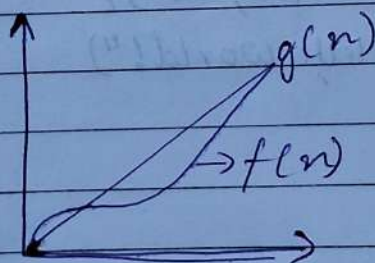
Best:- lowest time - omega Ω

Average:- average time - theta Θ

big O notation:

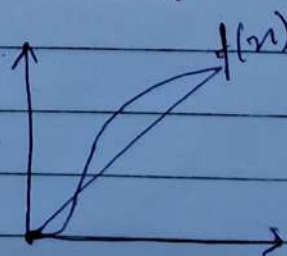
This measures the performance of an algorithm by simply providing the order of growth of the function.

This notation provides an upper bound on a function which ensures that function never grows faster than the upper bound.



Omega (Ω) notation

It basically describes best case scenario, which is opposite to big O notation. It is the formal way to represent lower bound of an algorithm's running time.



accesses the shared resources at a time

You are going to create traffic signal simulation by using threads.

DSA

Merging data:

$$O(m) + O(n) = O(m+n)$$

→ 2 pointer merge or linear merge

$n = [\quad]$

$m = [\quad]$

Foundation of complexity.

Writing a correct program answers only one question, A second equally important question is how efficiently the program reaches the answer as the input grows.

Complexity analysis is the discipline of answering that 2nd question before you ever run the code. Eg: Binary and linear search.

Two measures of complexity:-

Time →

Space → how memory grows as input grows.

Time complexity is not literally how many seconds it takes. It describes the growth rate of computational work relative to input size 'n'.

Thread lifecycle

Race condition:

A race condition occurs when multiple threads access shared mutable data concurrently, and the final result depends on the timing or order of their execution.

```
class counter {
    int count = 0;
    void increment () {
        count ++;
    }
}
```

Deadlock:

A deadlock occurs when 2 or more thread indefinitely for resources held by each other. If a race condition is a case of threads interfering with sharing or shared data, a deadlock is a case of threads interfering with each other's process.

Neither thread does anything wrong individually, but together they lock the program permanently.

Synchronization: it is a mechanism to control the access of multiple threads to share the resources. It is essential in multi thread that at environment to prevent thread interference and consistency problem. Synchronization ensures that only 1 thread can

Theta(Θ) notation:

This mainly describes average case

space complexity:

Only a fixed number of extra variables are used regardless of 'n'

additional space is big $O(1)$

Q Given numbers, find the indices of two numbers that add up to target.

Q write a program to calculate $a^b \bmod m$ using modular exponential, values of a & b maybe very large so that program should avoid directly calculating a^b

```
for (int i = -1; i <= 100; ++i) {  
    cout << "Hello World!"  
}
```

```
int target =
```

```
for (int i = 0; i < list.size(); i++) {  
    for (int j = i + 1; j < list.size(); j++) {  
        if (list.get(i) + list.get(j) == target) {  
            return i, j;  
        }  
    }  
}
```